NPS Prior Logistic Regression: Methods and MCMC Implementation

Ethan Pawl

November 2025

Overview

This R Markdown file describes the NPS prior approach to logistic regression specified in Salvatore et al. (2024), including the four priors

1. Mixed-Distance
2. Mixed-Distance-log
3. Mixed-Distance-log10
4. Power Prior

We present the model setup, prior definitions, the resulting Pólya–Gamma augmented full conditional distributions, and the MCMC algorithms implementing these methods.

Model Specification

We observe a probability sample (PS) $(y_i, \mathbf{x}_i)$ for $i = 1, \ldots, n$ and a nonprobability sample (NPS) $(\tilde{y}_\ell, \tilde{\mathbf{x}}_\ell)$ for $\ell = 1, \ldots, \tilde{n}$. The logistic model is

$$y_i \mid \boldsymbol{\beta} \sim \text{Bernoulli}(\theta_i), \qquad \theta_i = \frac{\exp(\mathbf{x}_i'\boldsymbol{\beta})}{1 + \exp(\mathbf{x}_i'\boldsymbol{\beta})}.$$

so each likelihood contribution is

$$p(y_i \mid \beta) = \frac{\exp(x_i'\beta)^{y_i}}{[1 + \exp(x_i'\beta)]}$$

Pseudolikelihood

Given weights $w_1, \ldots, w_n$ equal to the inverse probabilities of selection in the PS after rescaling to sum to the sample size n, we exponentiate the likelihood contributions as

$$\tilde{p}(y_i \mid \beta) = \frac{\exp(x_i'\beta)^{w_i y_i}}{[1 + \exp(x_i'\beta)]^{w_i}}$$

A likelihood-based model is recovered by setting each weight to 1.

Pólya–Gamma Data Augmentation

Each pseudolikelihood contribution can be rewritten as

$$\tilde{p}(y_i \mid \beta) = \frac{\exp(x_i'\beta)^{w_i y_i}}{[1+\exp(x_i'\beta)]^{w_i}} \propto \exp(\kappa_i^* x_i'\beta) \int_0^\infty \exp\left\{-\omega_i^* \frac{(x_i'\beta)^2}{2}\right\} p(\omega_i^*)$$

where $\kappa_i^* = w_i(y_i - \frac{1}{2})$ and $\omega_i^* \sim \text{PG}(w_i, 0)$.

Prior Specification

Mixed-Distance Priors: General Form

The general form is

$$\boldsymbol{\beta} \sim \mathcal{N}(\hat{\boldsymbol{\beta}}_{NPS}, \text{diag}(\sigma_0^2, \ldots, \sigma_p^2)).$$

This results in the full conditionals

$$\omega_i^* \mid \beta \sim PG(w_i^*, x_i'\beta), \qquad \beta \mid y, \omega^* \sim N(m_n^*, V_n^*),$$

where

$$V_n^* = (X'\Omega^* X + D_\sigma^{-1})^{-1}, \qquad m_n^* = V_n^*(X'\kappa^* + D_\sigma^{-1}\hat{\beta}_{NPS}),$$

$\Omega^* = \text{diag}(\omega_1^*, \ldots, \omega_n^*)$, $\kappa^* = w \circ (y - \frac{1}{2}1_n)$, and $D_\sigma = \text{diag}(\sigma_0^2, \ldots, \sigma_p^2)$. Here, w is the vector of all weights, $\circ$ denoted the elementwise (Hadamard) product, y is the vector of all PS observations, and 1_n is a length n vector where each entry is equal to 1.

Gibbs Sampler Implementation

```
mixed_dist_mcmc <- function(
    X,
    wts,
    PUMA_levels,
    prior_prec,
    beta_NP_hat,
    kappas,
    prior_name,
    niter,
    typeIerr
) {
    n <- nrow(X)
    p <- ncol(X)
    bet <- rep(0, p)
    Bet <- matrix(NA, p, niter)

    for(s in 1:niter) {
        # Polya-Gamma draws
        omega <- rpg(n, wts, as.numeric(X %*% bet))

        # Calculate posterior covariance and mean of coefficients
```

```
        post_prec <- crossprod(X * sqrt(omega)) + prior_prec

        # Get the precision Cholesky factor
        post_prec_L <- post_prec %>%
            chol() %>%
            t()

        # Invert for covariance Cholesky factor
        cov_L <- forwardsolve(post_prec_L, diag(p))
        post_mn <- as.numeric(
            tcrossprod(cov_L) %*%
            (crossprod(X, kappas) + prior_prec %*% beta_NP_hat)
        )

        # Draw by scaling and shifting standard normals
        bet <- as.numeric(cov_L %*% rnorm(p) + post_mn)
        Bet[,s] <- bet
    }

    U <- runif(n * niter) %>%
        matrix(n)
    P <- plogis(unname(X %*% Bet)) # expit of the linear predictor

    y_pred <- (U < P)
    storage.mode(y_pred) <- "integer"

    by_PUMA_sums <- y_pred %>%
        rowsum(group = PUMA, reorder = FALSE) # PUMA-level sums

    # Divide by # of individuals in each PUMA
    by_PUMA_means <- by_PUMA_sums / as.numeric(table(PUMA))[PUMA_levels]

    # Calculate area-level point and interval estimates
    point_est <- rowMeans(by_PUMA_means)
    CI <- rowQuantiles(by_PUMA_means, probs = c(typeIerr / 2, 1 - typeIerr / 2))

    res_df <- data.frame(
        PUMA = PUMA_levels,
        point_est = point_est,
        lower_CI = CI[,1],
        upper_CI = CI[,2],
        model = prior_name
    )

    return(res_df)
}
```

Power Prior

A power prior on β is

$$p(\beta \mid \tilde{y}, \tilde{X}) \propto L(\beta \mid \tilde{y}, \tilde{X})^a \, p(\beta),$$

with a fixed exponent a. The authors use a $t_3(0, 2.5)$ prior for each coefficient and we use the "scale mixture of normals" representation of the t distribution for conjugacy. Since the NPS is typically very large, we do not use Pólya-Gamma data augmentation since this would become a computational bottleneck. Instead, we use the conjugate update for τ_j^2

$$\tau_j^2 \sim \text{InvGamma}(3/2,\, 3(2.5)^2/2)$$

and a Metropolis-Hastings step for β to draw from the unnormalized full conditional

$$p(\beta \mid \text{data}, \{\tau_j^2\}) \propto \left\{ \prod_{i=1}^{n} \frac{\exp(x_i'\beta)^{w_i y_i}}{[1+\exp(x_i'\beta)]^{w_i}} \right\} \left\{ \prod_{\ell=1}^{\tilde{n}} \frac{\exp(\tilde{x}_\ell'\beta)^{a\tilde{y}_\ell}}{[1+\exp(\tilde{x}_\ell'\beta)]^{a}} \right\} \mathcal{N}(\beta \mid 0, \text{diag}(\tau_0^2, \ldots, \tau_p^2))$$

Gibbs Sampler (Remove this section later; for development only)

If we used Polya-Gamma data augmentation, the Gibbs sampler would be as follows (currently, we don't use this because it is too slow):

```
power_prior_gibbs <- function(
    X,
    X_NP,
    wts,
    a,
    PUMA_levels,
    kappas_all,
    niter
) {
    n <- nrow(X)
    n_NP <- nrow(X_NP)
    p <- ncol(X)

    X_all <- rbind(X, X_all)

    bet <- rep(0, p)
    Bet <- matrix(NA, p, niter)
    y_pred <- matrix(NA, niter, n)

    for(s in 1:niter) {
        # Latent variance draws
        tau2 <- 1 / rgamma(p, 2, (18.75 + bet^2) / 2)

        # PS Polya Gamma Draws
        omega <- rpg(n, wts, as.numeric(X %*% bet))
        # NPS Polya Gamma Draws
        omega_NP <- rpg(n_NP, a, as.numeric(X_NP %*% bet))

        # Calculate posterior covariance and mean of regression coefficients
        post_prec <- crossprod(X_all, diag(c(omega, omega_NP))) %*% X_all +
            diag(1 / tau2)

        post_cov <- post_prec %>%
            chol() %>%
            chol2inv()
```

```
        post_mn <- as.numeric(post_cov %*% crossprod(X_all, kappas_all))

        # Draw standard normals, scale and shift to the posterior distribution
        bet <- rmvnorm(1, post_mn, post_cov)[1,]
        Bet[,s] <- bet
    }

    U <- runif(n * niter) %>%
        matrix(n)
    P <- plogis(unname(X %*% Bet)) # expit of the linear predictor

    y_pred <- (U < P)
    storage.mode(y_pred) <- "integer"

    by_PUMA_sums <- y_pred %>%
        rowsum(group = PUMA, reorder = FALSE) # PUMA-level sums

    # Divide by # of individuals in each PUMA
    by_PUMA_means <- by_PUMA_sums / as.numeric(table(PUMA))[PUMA_levels]

    # Calculate area-level point and interval estimates
    point_est <- rowMeans(by_PUMA_means)
    CI <- rowQuantiles(by_PUMA_means, probs = c(typeIerr / 2, 1 - typeIerr / 2))

    res_df <- data.frame(
        PUMA = PUMA_levels,
        point_est = point_est,
        lower_CI = CI[,1],
        upper_CI = CI[,2],
        model = "NPS Prior: Power Prior"
    )

    return(res_df)
}
```

Metropolis-Hastings

Helper Functions First, the unnormalized full conditional posterior distribution of $\boldsymbol{\beta}$ is

```
pp_logpost <- function(
    bet,
    tau2,
    y,
    X,
    y_NP,
    X_NP,
    a,
    wts
) {
    eta <- X %*% bet
    eta <- as.numeric(eta)
    eta_NP <- X_NP %*% bet
    eta_NP <- as.numeric(eta_NP)
```

```
    loglik <- sum(wts * eta * y - wts * log(1 + exp(eta)))
    loglik_NP <- sum(a * eta_NP * y_NP - a * log(1 + exp(eta_NP)))
    prior_logprob <- sum(dnorm(bet, sd = tau2, log = TRUE))

    return(loglik + loglik_NP + prior_logprob)

}
```

We first use a naive (diagonal) proposal covariance matrix, run MCMC for some time, then use the empirical covariance of the $\boldsymbol{\beta}$ draws to specify the proposal covariance for the final model draws. The below function tunes the proposal covariance.

```
power_prior_mh_calibrate <- function(
    y,
    X,
    y_NP,
    X_NP,
    wts,
    a,
    niter = 200,
    init_scale = 1e-5,
    scaling_factor = 1,
    eps = 1e-8,
    maxTries = 10
) {
    n <- nrow(X)
    p <- ncol(X)

    bet <- rep(0, p)
    Bet <- matrix(NA, p, niter)

    prop_cov <- diag(init_scale^2, p)

    F_norm <- Inf
    tries <- 0
    while(tries < maxTries) {
        prop_cov_L <- prop_cov %>%
            chol()
        for(s in 1:niter) {
            # Latent variance draws
            tau2 <- 1 / rgamma(p, 2, (18.75 + bet^2) / 2)

            prop <- bet + prop_cov_L %*% rnorm(p)
            prop <- as.numeric(prop)

            log_accept_ratio <- pp_logpost(
                prop,
                tau2,
                y,
                X,
                y_NP,
                X_NP,
                a,
                wts
```

```
            ) -
            pp_logpost(
                bet,
                tau2,
                y,
                X,
                y_NP,
                X_NP,
                a,
                wts
            )

            bet <- if(log(runif(1)) < log_accept_ratio) prop else bet
            Bet[,s] <- bet
        }

        emp_cov <- cov(t(Bet))
        F_norm <- norm(emp_cov - prop_cov, "F")

        if(F_norm < eps & norm(emp_cov, "F") > 0) { # Covariance matrix has converged
            prop_cov <- emp_cov
            break
        } else { # Keep tuning
            if(norm(emp_cov, "F") == 0) {
                stop("Covariance is 0 matrix; bad initial guess")
            }
            new_diag <- scaling_factor * diag(emp_cov)
            prop_cov <- emp_cov
            diag(prop_cov) <- new_diag
            tries <- tries + 1
        }
    }

    if(tries == maxTries) stop("Max attempts reached before convergence.")

    return(prop_cov)
}
```

```
power_prior_mh <- function(
    y,
    X,
    y_NP,
    X_NP,
    wts,
    a,
    PUMA_levels,
    niter,
    typeIerr
) {

    n <- nrow(X)
    p <- ncol(X)
```

```r
bet <- rep(0, p)
Bet <- matrix(NA, p, niter)

prop_cov <- power_prior_mh_calibrate(y, X, y_NP, X_NP, wts, a)
prop_cov_L <- prop_cov %>%
    chol()

for(s in 1:niter) {
    # Latent variance draws
    tau2 <- 1 / rgamma(p, 2, (18.75 + bet^2) / 2)

    prop <- bet + prop_cov_L %*% rnorm(p)
    prop <- as.numeric(prop)

    log_accept_ratio <- pp_logpost(
        prop,
        tau2,
        y,
        X,
        y_NP,
        X_NP,
        a,
        wts
    ) -
    pp_logpost(
        bet,
        tau2,
        y,
        X,
        y_NP,
        X_NP,
        a,
        wts
    )

    bet <- if(log(runif(1)) < log_accept_ratio) prop else bet
    Bet[,s] <- bet
}

U <- runif(n * niter) %>%
    matrix(n)
P <- plogis(unname(X %*% Bet)) # expit of the linear predictor

y_pred <- (U < P)
storage.mode(y_pred) <- "integer"

by_PUMA_sums <- y_pred %>%
    rowsum(group = PUMA, reorder = FALSE) # PUMA-level sums

# Divide by # of individuals in each PUMA
by_PUMA_means <- by_PUMA_sums / as.numeric(table(PUMA))[PUMA_levels]

# Calculate area-level point and interval estimates
```

```
    point_est <- rowMeans(by_PUMA_means)
    CI <- rowQuantiles(by_PUMA_means, probs = c(typeIerr / 2, 1 - typeIerr / 2))

    res_df <- data.frame(
        PUMA = PUMA_levels,
        point_est = point_est,
        lower_CI = CI[,1],
        upper_CI = CI[,2],
        model = "NPS Prior: Power Prior"
    )

    return(res_df)
}
```

Metropolis-Hastings Implementation

MCMC Wrapper: All Prior Specifications

We use a wrapper function to call the above MCMC implementations for all four prior specifications. The three variants of the mixed-distance prior we use differ in how σ_j^2 is defined:

Mixed-Distance

$$\sigma_j = |\hat{\beta}_{PS,j} - \hat{\beta}_{NPS,j}|.$$

Mixed-Distance-log

$$\sigma_j = \sqrt{\frac{1}{\log \tilde{n}} \max\{(\hat{\beta}_{PS,j} - \hat{\beta}_{NPS,j})^2, \hat{\sigma}^2_{NPS,j}\}}.$$

Mixed-Distance-log10

$$\sigma_j = \sqrt{\frac{1}{\log_{10} \tilde{n}} \max\{(\hat{\beta}_{PS,j} - \hat{\beta}_{NPS,j})^2, \hat{\sigma}^2_{NPS,j}\}}.$$

The wrapper function follows.

```
nps_prior_mcmc <- function(
    y,
    X,
    y_NP,
    X_NP,
    niter,
    PUMA_levels,
    wts = NULL,
    typeIerr = 0.05
) {
    n <- length(y)
    p <- ncol(X)
    n_NP <- nrow(X_NP)
    if(is.null(wts)) { # If not using pseudolikelihood
        wts <- rep(1, n)
    } else { # If using pseudolikelihood
        stopifnot(
            "must provide one weight for every observation" =
```

```
            length(wts) == n
    )

    # Rescale the weights to sum to the sample size
    if(sum(wts) != n) {
        wts <- n * wts / sum(wts)
    }
}

# Get MLEs for regression coefficients from each sample
# PS
glm_fit <- glm(y ~ X - 1, binomial())
beta_hat <- coef(glm_fit) %>%
    as.numeric()

# NPS
glm_fit_NP <- glm(y_NP ~ X_NP - 1, binomial())
beta_NP_hat <- coef(glm_fit_NP) %>%
    as.numeric()

diff_vec <- beta_hat - beta_NP_hat
sq_dist <- diff_vec^2
kappas <- wts * (y - 0.5)

# Mixed-distance prior
prior_prec <- diag(1 / sq_dist)

md_res <- mixed_dist_mcmc(
    X,
    wts,
    PUMA_levels,
    prior_prec,
    beta_NP_hat,
    kappas,
    "NPS Prior: Mixed-Distance",
    niter,
    typeIerr
)

# for mixed-distance-log priors, we estimate the variance of the MLE
# of each coefficient based on the NPS
NP_vars <- glm_fit_NP %>%
    vcov() %>%
    diag() %>%
    as.numeric()

maxes <- pmax(sq_dist, NP_vars)

# Mixed-distance-log
prior_vars <- maxes / log(n_NP)
prior_prec <- diag(1 / prior_vars)

mdl_res <- mixed_dist_mcmc(
```

```
    X,
    wts,
    PUMA_levels,
    prior_prec,
    beta_NP_hat,
    kappas,
    "NPS Prior: Mixed-Distance-log",
    niter,
    typeIerr
)

# Mixed_distance-log10
prior_vars <- maxes / log10(n_NP)
prior_prec <- diag(1 / prior_vars)

mdl_ten_res <- mixed_dist_mcmc(
    X,
    wts,
    PUMA_levels,
    prior_prec,
    beta_NP_hat,
    kappas,
    "NPS Prior: Mixed-Distance-log10",
    niter,
    typeIerr
)

# Power Prior
# Set power prior exponent to p-value corresponding to Hotelling T^2 test
cov_beta_hat <- vcov(glm_fit)
t2 <- crossprod(diff_vec, (cov_beta_hat %>% chol() %>% chol2inv()) %*% diff_vec)
t2 <- as.numeric(t2)
f = p * (n - 1) / (n - p)
Fstat = t2 / f
a <- pf(Fstat, p, n - p, lower.tail = F)

# This would be for the power prior Gibbs sampler implementation (currently unused)
# kappas_all <- c(kappas, a * (y_NP - 1/2))

pp_res <- power_prior_mh(
    y,
    X,
    y_NP,
    X_NP,
    wts,
    a,
    PUMA_levels,
    niter,
    typeIerr
)

res_df <- rbind(
    md_res,
```

```
        mdl_res,
        mdl_ten_res,
        pp_res
    )

    return(res_df)
}
```

References

Salvatore, C., Biffignandi, S., Sakshaug, J., Wiśniowski, A., Struminskaya, B., "Bayesian Integration of Probability and Nonprobability Samples for Logistic Regression," Journal of Survey Statistics and Methodology, Volume 12, Issue 2, April 2024, Pages 458–492, https://doi.org/10.1093/jssam/smad041.